INDIA TAX ENGINE

Product review, strategic contradictions, technical risks & improvement roadmap

Purpose of this document

This document reviews the proposed **india-tax-engine** TypeScript npm package and turns the earlier critique into a practical decision document. It identifies what should remain, what should change, what should be added, what should be deferred, and what must be validated before the package is presented as reliable or expanded beyond India.

Core recommendation

Build the package, but position its first release as a narrowly scoped, deterministic and auditable Indian income-tax calculation engine—not as an automatically self-updating tax adviser. Make correctness, transparent rule provenance, explicit coverage limits and developer experience the differentiators.

Prepared from the supplied five-page product brief and the subsequent product critique. External competitor and market observations are treated as leads to re-verify, not as a complete market census.

PRODUCT DECISION DOCUMENT | NOVEMBER 2026

1. Executive assessment

The proposal has a sound technical foundation: a small TypeScript library, tax-year-specific rules stored as data, integer-paise money handling, step-by-step calculation output and old/new regime comparison. Those choices are worth retaining.

The main weakness is the gap between the promise and the mechanism. Versioned JSON files make rule changes easier to implement; they do not discover, interpret, verify or publish legislative changes automatically. The product needs a controlled rule-maintenance process around the calculation engine.

Dimension	Assessment	What it means
Technical learning value	High	Strong exercise in API design, exact arithmetic, testing, versioning and package publishing.
Developer usefulness	Promising	Useful where an application needs deterministic Indian tax calculations as a library rather than only a website calculator.
Correctness and maintenance burden	High risk	Tax logic must be sourced, tested, versioned and maintained as legislation changes.
Differentiation	Not yet proven	Explanation, JSON rules and regime comparison are valuable but are not individually unique.
Download growth	Uncertain	Downloads are not evidence of active adoption; distribution and repeat usage matter.
Global expansion	Long-term only	Shared infrastructure is possible, but legal rules and calculation paths remain jurisdiction-specific.

Recommended product promise

“For a supported taxpayer scenario and specified tax year, calculate tax using verified, versioned rules and return an explainable result that developers can test and audit.”

This is a measurable promise. It is safer and more credible than claiming that the package always knows the latest law or covers every taxpayer.

2. What the original proposal gets right

- **Rules as data:** Keep tax-year-specific rules separate from calculation code so changes can be reviewed and tested without rewriting the whole engine.
- **Exact money handling:** Use integer paise or another carefully specified exact representation; define safe ranges and statutory rounding at every relevant stage.
- **Deterministic calculation:** The same supported inputs and rule version should produce the same result. Do not rely on an LLM for arithmetic.
- **Explainable output:** Return calculation steps, not just a final number, so developers can debug and users can understand the result.
- **Regime comparison:** Comparing eligible regimes in one API call is a practical developer feature.
- **Small core:** A framework-independent package can serve Node.js, browser and serverless applications if its build and runtime compatibility are tested.

Keep these design choices. Improve the controls around them rather than replacing the core concept.

3. Strategic contradictions to resolve

Current assumption	Contradiction / weakness	Recommended correction
“Rules are in JSON, so the package stays updated.”	JSON is only a storage format. It cannot discover or verify new law.	Separate calculation engine, rule registry, change detection, human verification and release pipeline.
“A tax calculator is enough.”	A calculator is easy to copy and does not solve developer integration or legal traceability.	Compete on auditability, source-linked rules, stable APIs, tests and integration quality.
“More downloads mean a better package.”	Downloads may include CI, repeated installs and bots; they do not prove active use.	Measure dependents, repeat use, integrations, issues, successful installs and calculation agreement.
“The first release can serve everyone.”	Salary-only scope excludes many real scenarios; broad claims invite incorrect outputs.	Document a strict support matrix and reject unsupported cases explicitly.
“We can go global using the same tax logic.”	Tax concepts, tax periods, residency, deductions and rounding differ by country.	Share engineering infrastructure; isolate jurisdiction-specific rules and calculations.
“AI can automate tax updates.”	AI may misread provisions, effective dates or exceptions.	Use AI to flag and draft proposed changes; require authoritative review and tests before release.

4. Recommended product architecture

Treat the npm package as a deterministic calculation core surrounded by a controlled rules lifecycle. Avoid coupling the calculator to a web scraper, an AI model, a database service or a user interface.

Layer	Responsibility	Key design requirements
Public API	Accept structured taxpayer inputs and return typed results.	Stable types, clear defaults, validation errors, explicit supported scenarios.
Input validation	Check tax year, regime, taxpayer profile, amounts and combinations.	Reject invalid or unsupported combinations; do not silently ignore data.
Calculation core	Apply the tax rules in a deterministic order.	Pure functions where practical; explicit calculation stages; no network calls.
Rule registry	Provide the correct rule set for a tax period and taxpayer context.	Versioned, immutable releases; effective dates; jurisdiction and Act identifiers.
Rule provenance	Explain where each rule came from.	Official source URL, provision/section, publication date, effective date, review status.
Explanation layer	Produce machine-readable calculation traces and human-readable summaries.	Explain each applied, unavailable or rejected treatment; stable output schema.
Verification pipeline	Check rules and calculations before release.	Golden cases, boundary tests, regression tests, review checklist and signed release process.
Optional update service	Detect official publications and prepare change proposals.	Separate optional tooling; human approval; no silent runtime rule changes.

Important separation

The library should calculate offline from the rule set bundled with its version. A separate maintenance tool can monitor official publications and prepare a proposed update. This avoids a runtime calculation changing unexpectedly because an external website or service changed.

Suggested repository layout

india-tax-engine/ src/ api/ # public functions and types validation/ # input and support checks core/ # deterministic calculation stages money/ # exact arithmetic and rounding rules/ # rule types and registry interface explanations/ # structured calculation trace rules/ india/ 2025-26/ 2026-27/ tests/ unit/ boundary/ regression/ golden/ docs/ supported-scenarios.md sources-and-methodology.md release-process.md scripts/ validate-rules/ generate-change-report/ examples/ README.md

This is a suggested structure, not a requirement to create every folder in the first sprint.

5. API and output improvements

The original calculateTax({ taxYear, regime, income, deductions }) interface is a good starting point, but a single generic income field will become restrictive as more income heads and taxpayer profiles are introduced.

Illustrative input model

calculateTax({ jurisdiction: "IN", taxPeriod: "2026-27", regime: "new", taxpayer: { category: "individual", residentialStatus: "resident", ageBand: "below-60" }, income: { salary: 1200000, interest: 25000 }, deductions: { // Only explicitly supported deductions } })

Do not copy this API blindly. Confirm the legal inputs needed for every supported scenario before finalizing public types. Avoid asking for sensitive personal information that is not needed for calculation.

Recommended result shape

{ status: "complete", taxPeriod: "2026-27", jurisdiction: "IN", regime: "new", taxableIncome: 0, totalTax: 0, breakdown: [], appliedRules: [{ ruleId: "IN-...", ruleVersion: "..." }], assumptions: [], warnings: [], unsupportedItems: [] }

- Distinguish **complete**, **incomplete** and **unsupported** results; do not return a plausible total when material inputs are missing.
- Expose the exact rule-set version used so a past result can be reproduced.
- Return amounts in a clearly documented unit and representation. If the public API accepts rupees, define conversion and rounding; if it uses paise, make that unmistakable.
- Use stable error codes and typed warnings so applications can handle them programmatically.
- Keep human-readable explanation text separate from machine-readable calculation steps.

6. Scope: what to build now and what to defer

The original brief proposes resident salaried individuals, old and new regimes, tax years 2025-26 and 2026-27, slabs, standard deduction, rebate, marginal relief, cess and regime comparison. It defers capital gains, HRA, crypto, business income, NRIs and senior-citizen slabs. Retain a narrow first release, but describe it as a deliberately limited supported subset.

Capability	MVP decision	Reason / gate
Resident individual salary cases	Include	Core initial scenario; publish precise assumptions and exclusions.
Old/new regime comparison	Include where legally applicable	Use the same verified input facts and clearly report eligibility.
Tax years 2025-26 and 2026-27	Include only after verification	Keep distinct legal frameworks and effective periods; do not infer values.
Standard deduction, rebate, marginal relief, cess	Include after source verification	High-impact rules; test thresholds and interactions.
Surcharge	Do not imply support until complete	Either implement and test correctly or mark unsupported.
HRA and house-property income	Defer	Requires additional input structure and legal conditions.
Capital gains and special-rate income	Defer	May follow different rates and rules from ordinary slab income.
Business/professional income and presumptive taxation	Defer	Substantial additional rules and eligibility checks.
NRI and special residential categories	Defer	Residential status and special provisions materially affect results.
GST and corporate tax	Separate future products/modules	Different tax domains and users; avoid scope expansion in the first engine.

Never silently omit an income head or deduction. If a caller supplies an unsupported item, return a clear error or an incomplete result with an explicit warning.

7. Rule data: required metadata

A rule file should contain more than slab numbers. Every rule should be reviewable, traceable and tied to a precise period and scope.

- **Identity:** stable rule ID, jurisdiction, tax act, rule category and schema version.
- **Time:** tax period, effective-from date, effective-to date, announcement/publication date and supersession relationship.
- **Scope:** taxpayer category, residential status, regime and any eligibility conditions.
- **Legal source:** official URL, Act/section/rule/notification reference, source document date and relevant excerpt or citation pointer.
- **Verification:** draft/proposed/verified/retired status, reviewer, review date and change notes.
- **Calculation semantics:** amount/rate/threshold, ordering, interactions, caps, rounding and special-case behavior.
- **Integrity:** schema validation, checksum or immutable release reference.

Rule lifecycle

Stage	Action	Release condition
Detect	Monitor official Budget documents, Acts, rules, notifications and circulars.	A source publication is captured and dated.
Interpret	Identify affected provisions, periods and taxpayer categories.	A proposed rule change is documented.
Review	Compare against primary legal text and authoritative guidance.	A qualified reviewer approves interpretation.
Implement	Update rule data and, where needed, calculation logic.	Schema and calculation tests pass.
Verify	Run boundary, regression and independent reference cases.	No unexplained discrepancy remains.
Publish	Release a new immutable package/rule version and change log.	Release notes identify the affected tax periods and changes.

8. Testing and correctness strategy

For tax software, tests are not merely a quality feature. They are part of the product's trust proposition.

- **Golden/reference tests:** Hand-verified expected results with source citations and assumptions.
- **Boundary tests:** One rupee below, exactly at, and one rupee above every relevant threshold.
- **Interaction tests:** Rebate plus marginal relief, deductions plus regime eligibility, surcharge plus cess, and any other applicable combinations.
- **Historical regression tests:** A new tax-year rule release must not alter calculations for earlier periods.
- **Property tests:** Check invariants such as non-negative amounts where legally appropriate, consistent breakdown totals and deterministic output.
- **Rounding tests:** Test each legally required rounding stage; do not assume one final rounding operation is sufficient.
- **Cross-checks:** Compare with official calculator examples or independently verified calculations, documenting differences and assumptions.
- **Negative tests:** Invalid years, unsupported taxpayer types, contradictory inputs, unsafe magnitudes and missing mandatory facts.

Suggested release gate

Do not release a tax-year rules update until every rule has a source, every supported scenario has test coverage, threshold and interaction cases pass, historical tests remain stable, and the change has been reviewed. A green test suite is necessary but not sufficient if the expected values themselves were copied from an unreliable source.

9. Competitor and positioning review

The initial product brief names itr-agent, calcsuite-react, @mudraforge/statutory and GST-related packages. The earlier review also identified the official Income Tax Department calculator, india-finance-calc and other tax-calculator projects. This is a discovery list, not a complete or permanently current market census. Re-check npm, GitHub, package metadata, release dates, licenses, features and adoption before making public competitive claims.

Alternative / category	What it means for the product	Potential positioning
Official Income Tax Department calculator	Strong consumer reference; offers official calculation functionality.	Developer library and integration layer, not a replacement for the official portal.
itr-agent / AI-agent tools	Overlaps with deterministic tax calculations and regime comparison for agents.	Plain TypeScript library first; optional AI/MCP adapters later.
calcsuite-react / calculator collections	Overlaps with financial calculators and exact arithmetic.	Tax-focused rule provenance, coverage guarantees and auditable traces.
india-finance-calc / broad finance libraries	Shows that Indian finance functionality is not an empty category.	Depth and correctness in income tax rather than breadth of unrelated calculators.
Tax websites and filing platforms	Serve end users and may already provide a polished experience.	Power other products through an embeddable API; do not compete on consumer UX first.
GST utilities	Different tax domain with its own crowded ecosystem.	Keep GST outside the income-tax MVP.

Positioning statement to test

“A developer-first, auditable Indian income-tax calculation engine with versioned statutory rules, deterministic outputs and transparent calculation traces.”

Do not claim “first”, “only”, “most accurate” or “no competitors” without a reproducible, dated market audit and evidence.

10. Technical and operational risk register

Risk	Severity	Mitigation / decision
Incorrect tax calculation	Critical	Official primary sources, independent review, golden cases, boundary and interaction tests.
Missed law change or effective date	Critical	Source monitoring, explicit effective dates, review workflow and release checklist.
AI misinterpretation	High	AI may draft change proposals only; human approval required.
Unsupported income silently ignored	Critical	Strict schema, support matrix, explicit errors and incomplete-result status.
Rounding / integer overflow	High	Specify safe numeric bounds, exact arithmetic and statutory rounding tests.
API breaking changes	Medium	Semantic versioning, stable public types, migration guides and deprecation policy.
Bundle / runtime incompatibility	Medium	Test ESM/CJS strategy, Node versions, browser builds, TypeScript declarations and tree-shaking.
Dependency or supply-chain issue	Medium	Keep core dependencies minimal, pin development tooling and audit releases.
Low adoption	High	Developer interviews, examples, playground, integrations and usage metrics beyond downloads.
Legal / trust expectations	High	Clear scope and informational-use disclaimer; do not imply official endorsement or filing suitability.

11. Package optimization and developer experience

- **Small dependency-free core:** Keep optional integrations in separate entry points or companion packages.
- **Tree-shakeable builds:** Publish well-defined ESM and, if needed, CommonJS outputs; test actual consumer bundlers.
- **TypeScript quality:** Ship declarations, strict types, discriminated unions for result status and useful error types.
- **Stable public API:** Keep internal rule representation private where possible; expose only supported, documented contracts.
- **Runtime compatibility:** Test supported Node.js versions and browser environments; avoid accidental Node-only APIs in the core.
- **Performance:** Benchmark realistic workloads before optimizing. Tax calculations are unlikely to be CPU-heavy; correctness and bundle size matter more.
- **Documentation:** Include install, quick start, full input/output examples, supported-scenario matrix, limitations, source methodology and migration policy.
- **Accessibility of examples:** Provide JavaScript and TypeScript snippets, common framework integrations and a no-install playground.
- **Release hygiene:** Automated CI, tests, linting, package contents inspection, provenance where available, changelog and immutable releases.

12. Adoption, distribution and success metrics

Downloads are a useful visibility signal, not a quality score. npm download counts can include repeated installations and automation. Do not promise download targets until there is baseline evidence.

Metric	Why it matters	How to interpret it
npm downloads	Installation volume.	Trend over time; distinguish spikes from sustained usage where possible.
GitHub stars / forks	Initial interest and exploration.	Weak proxy; not proof of production usage.
Declared dependents	Other packages/apps reference it.	More meaningful than stars, but may still be inactive.
Repeat adoption / integrations	Developers continue using it.	Strong evidence of product value; collect ethically and transparently.
Install/build success	Package works in real projects.	Track CI and issue reports; test consumer environments.
Issue resolution and PRs	Community engagement and maintenance.	Look for substantive bug reports, fixes and external contributions.
Calculation agreement	Correctness against verified cases.	A core quality metric; document reference cases and discrepancies.
Rule update lead time	Maintenance responsiveness.	Measure from official publication to verified release, not merely automated detection.

Practical distribution plan

- Publish a polished README with a working 30-second quick start and explicit limitations.
- Provide examples for Node.js, Express, Next.js, browser apps and optional AI/MCP integration.
- Create an interactive playground that uses the same package version and displays the calculation trace.
- Write technical articles on difficult, verifiable topics such as marginal relief, tax-year transitions and exact rounding.
- Ask independent developers to review the API and verify selected test cases.
- Integrate the engine into PolicyLens AI as a real use case, but clearly distinguish a demo integration from independent adoption.
- Publish a clear contribution guide, issue templates, security reporting instructions and release notes.

13. Staged roadmap and decision gates

Phase	Build	Exit gate
0 — Validate	Re-check package name, npm/GitHub competitors, official tax sources, license and target developer needs.	A documented scope, source list, competitor snapshot and validated API design.
1 — MVP	Supported resident-salary scenarios; verified years/regimes; exact money; explanation; comparison; strict validation.	All advertised cases have source-linked tests; unsupported cases are rejected.
2 — Trust	Rule metadata, effective dates, provenance, golden tests, changelog and immutable rule versions.	A reviewer can reproduce a result and trace every material rule.
3 — Adoption	Stable package exports, ESM/CJS decision, types, CI, examples, playground and documentation.	Independent developers can install and integrate without direct help.
4 — Update workflow	Official-source monitoring, proposed diffs, human review, test generation and release automation.	A tax update can be processed safely and reproducibly.
5 — Expansion	Additional income heads and taxpayer categories, each with its own legal analysis and tests.	Each added category has an explicit support contract and verified reference cases.
6 — International	Add one new jurisdiction as a separate rule/calculation module.	Architecture supports the second jurisdiction without weakening the India implementation.

The original 12–13-hour estimate is suitable only as a prototype planning estimate. It should not be treated as a production readiness estimate for tax software.

14. Final build / no-build decision

Recommendation: **BUILD, with a narrower promise and stronger verification.** The package is worthwhile as a learning project, a reusable PolicyLens component and a potentially useful open-source developer tool. Commercial demand and download growth remain unvalidated.

Before coding or publishing, answer these questions:

- Can every supported tax value and condition be traced to an official source for the correct period?
- Can the team explain exactly which taxpayer scenarios are supported and which are rejected?
- Can historical results be reproduced after future rule updates?
- Are the difficult threshold and interaction cases independently verified?
- Can an external developer understand the API and integrate it without a personal walkthrough?
- Has the npm/GitHub landscape been re-checked, with competitor claims dated and evidence recorded?
- Is there an owner and process for maintaining rules after publication?

If the answer to source verification, supported-scope definition or maintenance ownership is no, do not describe the package as tax-accurate or automatically up to date. Build a prototype and label its limitations clearly.

15. Open questions to re-evaluate

- Should the package name remain india-tax-engine, or should the long-term brand leave room for multiple jurisdictions?
- Should rule files ship inside the package, or should a separate versioned rules package be used? For reproducibility and offline use, bundled immutable rules are the safer MVP default.
- What is the exact public money unit: rupees, paise, or a branded Money type?
- Which tax-year terminology and legal framework identifiers should the API expose during the transition?
- Will the first release support only salaried income, or also interest and other clearly defined income heads?
- What level of legal review is available, and who approves each rules release?
- Will the project remain community/open source, or eventually offer paid monitoring, hosted APIs or enterprise support?
- How will security, privacy and data minimization be handled if a future UI or hosted API accepts personal financial data?

16. Evidence, source boundaries and research notes

Source document: The supplied five-page “india-tax-engine: Product Brief and Build Decision” (November 2026). The original scope, architecture, proposed timeline, differentiators, risks and open items are taken from that document.

Subsequent product critique: The recommendations in this document synthesize the critique discussed after reviewing the brief. Architectural recommendations, product positioning, risk controls and roadmap sequencing are reasoned product recommendations, not statutory legal opinions.

External research observations: The earlier review identified the Income Tax Department's official calculator and Act resources, and discoverable projects including itr-agent, calcsuite-react, india-finance-calc and TaxCalc. These are competitor leads and must be rechecked for current functionality, publication status, downloads, licenses and maintenance. This document does not claim to be a complete competitor census or provide verified market-size statistics.

Primary-source rule: For actual tax values and interpretations, use the applicable official Act, Finance Act, rules, notifications, circulars and Income Tax Department materials. Blogs and search snippets may help discover an issue but should not be the final authority for rules data.

Legal boundary: This is a product and engineering review, not tax, legal or financial advice. It does not independently certify the numerical tax examples or legal interpretations in the original proposal. Verify all tax rules and example outputs with current primary sources and qualified review before release.

BOTTOM LINE

Build a deterministic Indian tax calculation engine first. Make each result traceable to an immutable, verified rule set. Treat automatic legal-change detection as a separate reviewed workflow. Win developer trust through correctness, clear limits, stable integration and reproducible tests—not by claiming automatic accuracy or chasing download counts.